

VÍDEO-MEMORIA DE ACTIVIDAD

Uso de Inteligencia Artificial en el Proyecto

LibraryReact — Librería Online

Asistente de IA utilizado:

Claude (Anthropic) — Modo Agente

Universidad Internacional de La Rioja (UNIR)
Grado en Ingeniería Informática — Desarrollo Web Full Stack

1. Introducción y contexto del proyecto

LibraryReact es una aplicación web de librería online desarrollada con React + Vite. El proyecto comprende autenticación de usuarios, catálogo de libros con filtros por género, carrito de la compra, detalle de libro, checkout y perfil de usuario con historial de pedidos.

Para el desarrollo de esta actividad se utilizó Claude (Anthropic) en modo agente como asistente de IA principal, siguiendo las recomendaciones de uso de GitHub Copilot PRO con modelos Claude para estudiantes de UNIR.

El asistente de IA se empleó específicamente para dos tareas obligatorias:

- **Estilizado completo de la aplicación: todo el código CSS para conseguir una línea visual coherente y atractiva.**
- **Generación de datos de prueba: batería completa de tests unitarios que cubre toda la lógica de la aplicación.**

2. Metodología de trabajo con la IA

Antes de comenzar cualquier tarea de código, se estableció un marco de trabajo estructurado con el agente. Esto supone una de las mejores prácticas en el uso profesional de IA: definir reglas de comportamiento explícitas desde el inicio de la sesión.

2.1 Establecimiento del entorno de trabajo

El primer paso fue crear un archivo CLAUDE.md con reglas de desarrollo que el agente debía leer y respetar en cada ejecución:

Prompt 1 — Inicialización del agente con reglas de desarrollo

Lee toda la estructura del proyecto y su lógica. Genera un CLAUDE.md con la información que concluyas. Dentro del archivo define las siguientes reglas de desarrollo que leerás cada vez que te ejecutes: Regla 1: No se puede modificar la lógica del proyecto a no ser que sea estrictamente necesario por una dependencia. Regla 2: Cada cambio que hagas debes pedir primero aprobación. Regla 3: Tienes que leer de la carpeta /skills todas las skills antes de intentar hacer un cambio en el proyecto.

 **Resultado:** Claude generó el CLAUDE.md con la documentación técnica completa del proyecto (stack, rutas, contextos AuthContext y CartContext, modelos de datos) y las 3 reglas de desarrollo establecidas.

2.2 Creación del sistema de skills especializadas

Las skills son documentos de referencia que el agente debe consultar antes de actuar, simulando el criterio de un desarrollador senior especializado en cada área:

Prompt 2 — Creación de skills especializadas

Crea la carpeta /skills y dentro crea las siguientes skills:• skill-frontend-senior.md — criterios de calidad React: estructura de componentes, uso correcto de hooks, separación de responsabilidades.• skill-css-senior.md — criterios para CSS: convenciones de nomenclatura, layout, responsive, variables CSS.• skill-qa-senior.md — criterios para testing frontend: qué cubrir, cómo abordar casos límite, herramientas compatibles con el stack actual.

 Resultado: 3 archivos skill creados en /skills con criterios profesionales para React, CSS y QA. El agente los consultó automáticamente antes de cada cambio posterior.

3. Tarea 1 — Estilizado de la aplicación

Con la base del proyecto establecida (hooks y componentes ya creados sin tocarlos), se procedió a generar todos los estilos CSS de la aplicación.

3.1 Propuesta inicial y decisión arquitectónica

Prompt 3 — Solicitud de estilos completos

Tenemos ya la base de todo el proyecto, toda la lógica de los hooks y los componentes están creados, no hay que tocarlos. Como ya tienes el contexto necesario del proyecto, quiero que generes los estilos de toda la aplicación. Necesito que con tus skills de front developer y modelado me vayas mostrando las mejoras que crees convenientes: estilos, ordenación, dónde colocar las cosas, etc.

La IA propuso inicialmente crear archivos CSS junto a cada componente (patrón colocation). El desarrollador cuestionó esta decisión, lo que generó un debate técnico productivo:

Prompt 4 — Cuestionamiento de la arquitectura CSS (interacción clave)

No crees que es mejor poner los CSS todos juntos en algún lugar? Los estás poniendo dentro de pages/ y components/. ¿Crees que es la mejor manera?

 **Resultado:** La IA reconoció la validez del argumento y propuso una arquitectura centralizada en `src/styles/` con 4 archivos temáticos: `global.css`, `layout.css`, `components.css` y `pages.css`. Esta es la solución implementada finalmente.

3.2 Estructura CSS final generada

Tras la aprobación de la nueva arquitectura, se generó el sistema completo:

```
src/
├─ index.css           → único punto de entrada (4 @import)
├─ styles/
│   ├─ global.css      → variables CSS, reset, estilos base
│   ├─ layout.css      → header, page-layout, columnas, responsive
│   ├─ components.css  → BookCard, Cart, GenreFilter, SearchBar
│   └─ pages.css       → Landing, Login, Home, BookDetail, Checkout, Profile
```

3.3 Iteraciones y correcciones

No todas las respuestas fueron correctas a la primera. Se registraron al menos dos iteraciones importantes:

Prompt 5 — Corrección de alineación del carrito

Me gusta lo que hicistes, lo único que te diría que cambies es que en el detalle del libro el carrito con el checkout quede en el mismo sitio de la derecha que la home.

La IA realizó el cambio pero no funcionó correctamente. Fue necesaria una segunda iteración hasta conseguir que flex: 1 en el elemento <main> de BookDetail empujara el carrito al borde derecho correctamente.

Prompt 6 — Rediseño del layout de BookDetail

Ahora sí funciona, pero no me gusta el estilo de la página del detalle del libro, hazla más profesional. Solo está la foto y los datos debajo, ponlo mejor: imagen izquierda centrada y los datos a su derecha.

 **Resultado:** La IA propuso y ejecutó (previa aprobación) añadir dos divs wrapper en BookDetail.jsx para agrupar imagen e info en layout horizontal: imagen fija a 220px a la izquierda, datos con separadores tipo ficha de producto a la derecha. Responsive en móvil.

4. Tarea 2 — Generación de tests unitarios

Una vez completado el estilizado, se solicitó la generación de una batería completa de tests que cubriera toda la lógica de la aplicación.

Prompt 7 — Solicitud de batería completa de tests

Listo, tenemos la aplicación. Ahora quiero que generes una batería de tests unitarios completa que compruebe toda la lógica de la aplicación.

El agente consultó primero la skill-qa-senior.md y presentó un plan detallado para aprobación antes de escribir ninguna línea de código:

4.1 Plan propuesto y aprobado

Archivo de test	Qué cubre
<code>AuthContext.test.jsx</code>	login/logout, persistencia en localStorage, estado inicial
<code>CartContext.test.jsx</code>	addToCart, incremento cantidad, removeFromCart, clearCart, addOrder
<code>booksApi.test.js</code>	getBooks, getBookById (ID válido, string, inexistente)
<code>ProtectedRoute.test.jsx</code>	redirección sin usuario, render con usuario autenticado
<code>Header.test.jsx</code>	estado sin/con usuario, nombre de la app
<code>BookCard.test.jsx</code>	title, author, code, price
<code>Cart.test.jsx</code>	carrito vacío, total unidades, botón checkout
<code>GenreFilter.test.jsx</code>	render géneros, check/uncheck, callback
<code>LandingPage.test.jsx</code>	botones sin usuario, redirección con usuario
<code>LoginPage.test.jsx</code>	formulario, error credenciales, login correcto, localStorage
<code>CheckoutPage.test.jsx</code>	carrito vacío, total, alert, clearCart, orders, navegación

4.2 Resultado final

Tras la aprobación del plan, se generaron los 11 archivos de test con resultado:

Área	Tests	Estado
Contextos (Auth + Cart)	18	✓ Pasando
API y utilidades	5	✓ Pasando

Componentes UI	15	✓ Pasando
Páginas y rutas	31	✓ Pasando
TOTAL	69	✓ 69/69

5. Métricas de uso de la IA

5.1 Porcentaje de respuestas correctas e incorrectas

Métrica	Valor
Respuestas correctas a la primera	~75%
Respuestas parcialmente correctas (1-2 iteraciones)	~15%
Respuestas incorrectas o que requirieron rehacerse	~10%

Las respuestas que requirieron iteración correspondieron principalmente al ajuste del carrito en BookDetail (2 rondas de corrección) y al layout vertical/horizontal en la página de detalle del libro. Las respuestas correctas a la primera incluyeron toda la arquitectura CSS centralizada, la batería completa de tests y las skills.

5.2 Líneas de código generadas con IA

Métrica	Valor
global.css + layout.css + components.css + pages.css	~450 líneas
11 archivos de test (69 tests)	~650 líneas
CLAUDE.md + 3 skills en /skills	~150 líneas
Cambios en JSX (classNames + wrappers)	~25 líneas
TOTAL ESTIMADO	~1.275 líneas

5.3 Estimación del tiempo ahorrado

Tarea	Tiempo manual	Con IA	Ahorro
Sistema CSS completo (4 archivos)	6-8 horas	~30 min	~7 horas
Batería de 69 tests unitarios	8-10 horas	~20 min	~9 horas
CLAUDE.md + 3 skills	2-3 horas	~10 min	~2.5 horas

TOTAL	16-21 horas	~1 hora	15-20 horas
--------------	--------------------	----------------	--------------------

La estimación se basa en comparar el tiempo de desarrollo manual (investigación de selectores, escritura de tests caso a caso, documentación desde cero) frente al tiempo real empleado en redactar prompts y revisar los resultados generados.

6. Prompts destacados y análisis crítico

A continuación se detallan los prompts más representativos de la sesión, seleccionados por su impacto técnico o por ilustrar el tipo de interacción correcta con un agente de IA:

Prompt más valioso — Arquitectura CSS

Cuestionamiento de la solución inicial

No crees que es mejor poner los CSS todos juntos en algún lugar? Los estás poniendo dentro de pages/ y components/. ¿Crees que es la mejor manera?

Este prompt es el más relevante de la sesión porque demuestra el uso crítico de la IA: no aceptar la primera propuesta, sino cuestionar las decisiones arquitectónicas. La IA reconoció que su propuesta inicial (colocation) era funcional pero no la mejor para el tamaño del proyecto, y propuso la solución centralizada que se implementó.

Prompt de control — Reglas de desarrollo

Establecimiento del marco de trabajo

Genera un CLAUDE.md con las siguientes reglas: (1) No modificar lógica existente salvo dependencia crítica. (2) Pedir aprobación antes de cada cambio. (3) Leer /skills antes de actuar.

Establecer reglas explícitas al inicio de una sesión de agente es una práctica profesional clave. Permitió que todos los cambios posteriores fueran predecibles, reversibles y documentados.

Prompt de iteración — Corrección de UI

Corrección tras resultado insatisfactorio

Ahora sí funciona, pero no me gusta el estilo de la página del detalle del libro. Hazla más profesional: imagen izquierda y los datos a su derecha.

Este prompt ilustra que la IA no siempre genera el resultado esperado a la primera, especialmente en tareas de diseño visual. La iteración y la descripción precisa del resultado deseado son imprescindibles.

7. Conclusiones

El uso de Claude en modo agente para el desarrollo de LibraryReact ha demostrado ser altamente productivo en las tareas específicas asignadas. Las principales conclusiones son:

- **La IA es especialmente eficaz para tareas de volumen repetitivo: generar decenas de tests siguiendo un patrón, o escribir centenares de líneas de CSS con coherencia visual.**
- **El valor real del agente se maximiza cuando el desarrollador mantiene criterio propio: cuestionar propuestas, pedir alternativas y establecer reglas de comportamiento explícitas.**
- **Las respuestas incorrectas o parciales (25%) son esperables y no suponen un problema si se identifican en la revisión. El flujo correcto es: revisar → iterar → aprobar.**
- **El ahorro estimado de 15-20 horas en tareas de CSS y testing permitió dedicar ese tiempo a la lógica de negocio y la arquitectura del proyecto, que es donde el criterio humano es irremplazable.**

En resumen: la IA no reemplaza al desarrollador, pero multiplica su capacidad de ejecución en tareas bien definidas. La clave está en saber qué delegar y cómo supervisar el resultado.